

Práctica

Objetivo de la Actividad

Aplicar los conocimientos adquiridos sobre el patrón arquitectónico **Modelo-Vista-Controlador (MVC)**, enrutamiento con Express y operaciones CRUD utilizando la librería `mysql2`.

En esta ocasión, deberás replicar la estructura lógica que desarrollamos para la entidad *Productos*, pero adaptándola por completo para gestionar el ciclo de vida de la entidad **Usuarios**.

1. Estructura de la Base de Datos

Para este ejercicio, trabajarás con la tabla `usuarios`. Asegúrate de ejecutar el siguiente script en tu cliente de base de datos (MySQL Workbench, phpMyAdmin, etc.) antes de comenzar a programar:

```
CREATE TABLE `usuarios` (  
  `id` int NOT NULL AUTO_INCREMENT,  
  `nombre` varchar(45) NOT NULL,  
  `edad` int DEFAULT NULL,  
  `altura` decimal(5,2) DEFAULT NULL,  
  `correo` varchar(45) DEFAULT NULL,  
  `empresa_id` int DEFAULT NULL,  
  `foto_url` varchar(100) DEFAULT NULL,  
  PRIMARY KEY (`id`),  
)
```

2. Instrucciones del Desafío

Deberás crear y configurar los siguientes componentes en tu proyecto de Node.js:

A. El Modelo (`models/usuarioModel.js`)

Crea un objeto contenedor llamado `Usuario` que interactúe con la base de datos mediante la librería `mysql2`. Este modelo debe incluir las siguientes funciones de consulta:

- `obtenerTodos(callback)`: Ejecuta un `SELECT *` de la tabla `usuarios`.

- `obtenerPorId(id, callback)`: Busca un usuario específico mediante su ID (`WHERE id = ?`).
- `crear(datos, callback)`: Inserta un nuevo registro. Recuerda omitir el campo `id` ya que la base de datos lo genera automáticamente (`AUTO_INCREMENT`).
- `actualizar(id, datos, callback)`: Modifica los campos de un usuario basándose en su ID.
- `eliminar(id, callback)`: Remueve permanentemente un usuario de la tabla.

B. El Controlador (`controllers/usuariosController.js`)

Desarrolla las funciones que recibirán las peticiones HTTP (`req`) y enviarán las respuestas al cliente (`res`):

1. **Validación Obligatoria:** Crea una función auxiliar interna para validar que el cuerpo de la petición (`req.body`) contenga obligatoriamente el campo `nombre` (ya que es el único definido como `NOT NULL` en la tabla). Si falta, debe responder inmediatamente con un estado `400 Bad Request`.
2. **Control de Flujo:** Asegúrate de colocar la palabra clave `return` dentro de los bloques de evaluación de errores (`if (err)`). Esto evitará el colapso del servidor por intentar enviar respuestas duplicadas (`ERR_HTTP_HEADERS_SENT`).
3. **Manejo de Errores Reales:** Al capturar un error de MySQL, imprime detalladamente el objeto `err` en la consola del servidor usando `console.error(err)` para facilitar el diagnóstico durante tus pruebas.

C. El Enrutador (`routes/usuarios.js`)

Configura un enrutador de Express (`express.Router()`) que conecte los métodos HTTP y las URLs con las funciones correspondientes del controlador:

- `GET /` Obtener todos los usuarios.
- `GET /:id` Obtener un usuario por ID.
- `POST /` Crear un nuevo usuario.
- `PUT /:id` Actualizar un usuario existente.
- `DELETE /:id` Eliminar un usuario.

D. El Servidor Principal (`app.js`)

Importa tu nuevo enrutador de usuarios y móntalo globalmente bajo el prefijo de ruta `/usuarios`.

3. Guía de Pruebas en Postman (Criterios de Aceptación)

Una vez finalizado el código, comprueba el correcto funcionamiento de tu API simulando las peticiones en Postman utilizando el protocolo **HTTP estándar** (<http://localhost:3000/usuarios>).

Para probar la inserción (**POST**), configura el cuerpo en modo **raw / JSON** y envía la siguiente estructura de datos de ejemplo:

```
{
  "nombre": "Ana María Restrepo",
  "edad": 28,
  "altura": 1.68,
  "correo": "ana.restrepo@email.com",
  "empresa_id": null,
  "foto_url": "[https://miweb.com/fotos/ana.jpg](https://miweb.com/fotos/ana.jpg)"
}
```

Resultados Esperados en Postman:

- **POST:** Debería retornar un estado **201 Created** junto con el texto "Usuario creado con ID: [Número]".
- **GET (Todos):** Debería retornar un estado **200 OK** y una lista en formato JSON que contenga al usuario que acabas de crear.
- **GET (Por ID):** Al consultar `/usuarios/1` (o el ID generado), debería devolver únicamente el objeto de ese usuario.